

在数据库应用系统的开发中,需要使用编程方法对数据库进行操纵。本章从两个方面讲解数据库编程技术:一是扩展 SQL 自身的功能;二是在高级语言程序中使用 SQL,实现高级语言与数据库之间的交互,开发应用需求中复杂业务逻辑的技术和方法。

8.1 基本知识点

本章讲解开发数据库应用系统的编程技术,内容的实践性较强。希望读者多上机实验,通过动手实践,掌握数据库编程方法和技术,提高开发数据库复杂应用的能力。

① 需要了解的:了解 SQL 表达能力的限制,以及掌握数据库编程技术的必要性。数据库编程技术可以有效弥补 SQL 无法实现复杂业务逻辑的不足,提高应用系统和 RDBMS 间的互操作性。

② 需要牢固掌握的:掌握新引入的 SQL 子句(例如 WITH RECURSIVE 子句)、新的内置函数;掌握 PL/SQL 与存储过程/存储函数的基本概念、基本结构、语句语法和用法;掌握游标的概念和使用方法;掌握在 Java 程序中使用 JDBC 访问数据库的方法。

③ 需要举一反三的:能够在实际安装的 RDBMS 上通过编程的方法开发应用程序,完成对数据库的各种操作;能够使用 JDBC 来进行数据库应用程序的设计,使设计的应用系统可移植性好,并且能同时访问不同的数据库,共享数据资源。

8.2 习题解答和解析

1. 假定一门课程存在多门直接先修课程,使用 WITH RECURSIVE 子句查找“数据库系统概论”课程的所有先修课课程号和课程名称。

【解析】请读者阅读《概论》第8章 8.1.2 小节“扩展 SQL 的功能”相关内容。

本题假定一门课程存在多门直接先修课程,例如“数据库系统概论”的先修课有“数据结构”和“操作系统”,“操作系统”的先修课有“程序设计基础与 C 语言”和“离散数学”。所以,我们先要在原来的 Course 表中插入相应的记录,如下表所示。当然我们首先要修改 Course 表的关系模式,因为课程号不再是主码了。请读者思考一下,这时 Course 表的主码是什么?

Course 表是第几范式？应该如何优化 Course 表为 BCNF？

课程号 Cno	课程名 Cname	学分 Ccredit	先修课 Cpno
81001	程序设计基础与 C 语言	4	
81002	数据结构	4	81001
81003	数据库系统概论	4	81002
81004	信息系统概论	4	81003
81005	操作系统	4	81001
81006	Python 语言	3	81002
81007	离散数学	4	
81008	大数据技术概论	4	81003
81003	数据库系统概论	4	81005
81005	操作系统	4	81007

本题相应代码如下：

```
WITH RECURSIVE RS AS
( SELECT Cpno FROM Course WHERE Cname = '数据库系统概论'
  /*初始化 RS，假设结果集为 L[1]，即“数据库系统概论”的所有先修课*/
  UNION
  SELECT Course.Cpno FROM Course, RS WHERE RS.Cpno = Course.Cno
) /*递归查询第 i 层(i ≥ 1)的数据，即第 i-1 层数据的先修课课程号，并更新 RS */
SELECT Cno, Cname FROM Course WHERE Cno IN (SELECT Cpno FROM RS);
/*根据 RS 中记录的所有先修课程号，通过查找课程表，输出课程号与课程名*/
```

2. 对“学生选课”数据库编写存储过程，完成下述功能：

- ① 统计“离散数学”课程的成绩分布情况，即按照各分数段统计人数。
- ② 统计任意一门课的平均成绩。
- ③ 将学生选课成绩从百分制改为等级制（即 A、B、C、D、E）。

答：

①

```
CREATE PROCEDURE discrete_math_grade()
AS
DECLARE CURSOR dist FOR          /*定义游标*/
SELECT grade FROM SC WHERE cno =
    (SELECT Cno FROM Course WHERE Cname = '离散数学');
p_100    NUMBER := 0;
```

```
p_90    NUMBER := 0;
p_80    NUMBER := 0;
p_70    NUMBER := 0;
p_60    NUMBER := 0;
p_others NUMBER := 0;
p_grade NUMBER;
BEGIN
  OPEN dist;                                /*打开游标*/
  LOOP
    FETCH dist INTO p_grade;                /*使用游标*/
    EXIT WHEN (dist%NOTFOUND);
    IF (p_grade = 100) THEN
      p_100 := p_100 + 1;
    ELSIF (p_grade >= 90) THEN
      p_90 := p_90 + 1;
    ELSIF (p_grade >= 80) THEN
      p_80 := p_80 + 1;
    ELSIF (p_grade >= 70) THEN
      p_70 := p_70 + 1;
    ELSIF (p_grade >= 60) THEN
      p_60 := p_60 + 1;
    ELSE
      p_others := p_others + 1;
    END IF;
  END LOOP;
  CLOSE dist;                                /*关闭游标*/
END;
```

- ② 【解析】输入任意一门课的课程名，计算选修该课程学生的平均成绩。

```
CREATE PROCEDURE avegrade(incname CHAR(40))
AS
BEGIN
  SELECT AVG(Grade) FROM SC WHERE Cno =
    (SELECT Cno FROM Course WHERE Cname = incname);
END;
```

- ③ 【解析】扫描 SC 表，把学生选课成绩 grade 的值由百分制更新为等级制。

```
CREATE PROCEDURE gradetype()
AS
  DECLARE CURSOR gradecursor FOR SELECT grade FROM SC;
```

```

        scgrade    NUMBER;
        score      CHAR(1);
BEGIN
    OPEN gradecursor;                                /*打开游标*/
LOOP
    FETCH gradecursor INTO scgrade;                    /*使用游标*/
    EXIT WHEN (gradecursor %NOTFOUND);
    IF (scgrade >=90 AND scgrade <=100) THEN
        score := 'A';
    ELSIF (scgrade >= 80 AND scgrade <90) THEN
        score := 'B';
    ELSIF (scgrade >= 70 AND scgrade <80) THEN
        score := 'C';
    ELSIF (scgrade >= 60 AND scgrade <70) THEN
        score := 'D';
    ELSE
        score := 'E';
    END IF;
END LOOP;
END;
```

3. 设有产品表的关系模式 Product(PID, PName, Price, Amount, DateOfManufacture)，描述的是产品编号、产品名称、单价、数量、生产日期，其中主码为产品编号与生产日期。请基于某一种 RDBMS，使用 DDL 语言完成 Product 关系模式的创建，并使用存储过程或存储函数生成不少于 1 000 条产品记录。

【解析】以下代码基于 mysql 数据库：

```

/* 当自动生成记录数量较大时，首先创建内存表可以提高效率 */
CREATE TABLE product_mem (
    PID int(11) NOT NULL AUTO_INCREMENT comment '产品编号',
    PName varchar(30) NOT NULL comment '产品名称',
    Price decimal(5, 2) NOT NULL comment '单价',
    Amount integer NOT NULL comment '数量',
    DateOfManufacture date NOT NULL comment '创建时间',
    PRIMARY KEY (PID, DateOfManufacture)
) ENGINE=MEMORY DEFAULT CHARSET=utf8mb4;

/* 再创建普通表，存储产品记录 */
CREATE TABLE Product (
```



```
PID int(11) NOT NULL AUTO_INCREMENT comment '产品编号',
PName varchar(30) NOT NULL comment '产品名称',
Price decimal(5, 2) NOT NULL comment '单价',
Amount integer NOT NULL comment '数量',
DateOfManufacture date NOT NULL comment '创建时间',
PRIMARY KEY (PID, DateOfManufacture)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

/*生成由 1~9 组成的字符串, 输入参数 n 为返回字符串的长度, 用于生成随机数量和价格*/
DELIMITER $$
CREATE FUNCTION randNum(n int) RETURNS VARCHAR(255)
BEGIN
    DECLARE chars_str varchar(20) DEFAULT '123456789';
    DECLARE return_str varchar(255) DEFAULT "";
    DECLARE i INT DEFAULT 0;
    WHILE i < n DO
        SET return_str = concat(return_str, substring(chars_str,
            FLOOR(1 + RAND()*9), 1));
        SET i = i + 1;
    END WHILE;
    RETURN return_str;
END $$
DELIMITER ;

/* 生成由大小写字母 a~z 和数字 0~9 组成的字符串,
输入参数 n 为返回字符串的长度, 用于生成随机 PName */
DELIMITER $$
CREATE FUNCTION randStr(n INT) RETURNS varchar(255) CHARSET utf8mb4
DETERMINISTIC
BEGIN
    DECLARE chars_str varchar(100) DEFAULT
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';
    DECLARE return_str varchar(255) DEFAULT "";
    DECLARE i INT DEFAULT 0;
    WHILE i < n DO
        SET return_str=concat(return_str, substring(chars_str,
            FLOOR(1+RAND()*62), 1));
        SET i = i + 1;
    END WHILE;
    RETURN return_str;
END $$
DELIMITER ;
```

```
        END WHILE;
    RETURN return_str;
END$$
DELIMITER ;

/* 创建存储过程，向内存表中插入数据 */
DELIMITER $$
CREATE PROCEDURE add_product_mem(IN n int)
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE (i <= n) DO
        INSERT INTO product_mem (PName, Price, Amount, DateOfManufacture)
            VALUES (randStr(20), CONCAT(randNum(3), ".", randNum(2)),
                randNum(2), curdate());
        SET i = i + 1;
    END WHILE;
END $$
DELIMITER ;

/* 调用循环来生成数据，输入参数 n 为循环次数，count 为每次循环产生的数据量 */
DELIMITER $$
CREATE PROCEDURE add_Product(IN n int, IN count int)
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE (i <= n) DO
        CALL add_product_mem(count);
        INSERT INTO Product SELECT * FROM product_mem;
        delete from product_mem;
        SET i = i + 1;
    END WHILE;
END $$
DELIMITER ;

/* 如果生成 1000 条记录，1000 = 10 * 100；调用 add_Product() */
CALL add_Product(10, 100);
```

4. 使用 JDBC 编写应用程序，实现对异构数据库中的数据进行迁移。要求：

- ① 配置两个不同的数据源，编写程序连接两种不同 RDBMS 的数据源。
- ② 分别在上述两种 RDBMS 中完成第 3 题关系模式 Product 的创建及产品记录的生成。

③ 使用 JDBC 编写应用程序,将其中一种 RDBMS 中产品表的数据迁移到另一种 RDBMS 中的产品表进行追加。

/* 此处配置的两个数据源为 MySQL 和 PostgreSQL。MySQL 中创建关系模式及记录的生成已在上面第 3 题完成,接下来为 PostgreSQL 数据库创建表 */

```
CREATE TABLE Product (  
    PID serial NOT NULL,  
    PName varchar(30) NOT NULL ,  
    Price decimal(5, 2) NOT NULL ,  
    Amount integer NOT NULL ,  
    DateOfManufacture date NOT NULL ,  
    PRIMARY KEY (PID, DateOfManufacture)  
);
```

/* 在 Eclipse 开发平台中创建 Java 项目,根据 MySQL 和 PostgreSQL 各自数据库环境的参数配置修改如下代码,如数据库名称、表名、账号和密码等。连接两个数据库,实现数据迁移 */

```
package jdbc_test;  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.sql.PreparedStatement;  
import java.sql.Statement;  
public class conn_pg {  
    public static void main(String[] args) {  
        Connection connection_pg=null;  
        Statement statement_pg =null;  
        Connection connection_mysql=null;  
        Statement statement_mysql =null;  
        PreparedStatement preparedStatement = null;  
        String sql_add = null;  
        try{  
            int i = 0;  
            String url_pg="jdbc:postgresql://127.0.0.1:5432/pg_db";  
            String user_pg="ctf";  
            String password_pg = "1";  
            Class.forName("org.postgresql.Driver");  
            connection_pg= DriverManager.getConnection(url_pg, user_pg,
```

```
        password_pg);
System.out.println("成功连接 pg 数据库");
sql_add = "insert into Product (pname, price, amount, dateofmanufacture)
        values (?, ?, ?, ?)";
preparedStatement = connection_pg.prepareStatement(sql_add);

String url_mysql="jdbc:mysql://127.0.0.1:3306/mysql_db";
String user_mysql="root";
String password_mysql = "rootroot";
connection_mysql= DriverManager.getConnection(url_mysql, user_mysql,
        password_mysql);

System.out.println("成功连接 mysql 数据库");
String sql_mysql="select * from Product";
statement_mysql=connection_mysql.createStatement();
ResultSet resultSet_mysql=statement_mysql.executeQuery(sql_mysql);
while(resultSet_mysql.next()){
    i++;
    preparedStatement.setString(1, resultSet_mysql.getString(2));
    preparedStatement.setBigDecimal(2, resultSet_mysql.getBigDecimal(3));
    preparedStatement.setInt(3, resultSet_mysql.getInt(4));
    preparedStatement.setDate(4, resultSet_mysql.getDate(5));
    preparedStatement.addBatch();

    /* 积攒够 100 条记录，就批量更新 */
    if(i % 100 == 0)
    {
        preparedStatement.executeBatch();
        preparedStatement.clearBatch();
    }
}

preparedStatement.executeBatch();
preparedStatement.clearBatch();

System.out.println("mysql->pg:数据迁移完毕!");
```



```
        } catch (Exception e) {
            throw new RuntimeException(e);
        } finally {
            try {
                statement_pg.close();
                statement_mysql.close();
                preparedStatement.close();
            }

            catch (SQLException e) {
                e.printStackTrace();
                throw new RuntimeException(e);
            } finally {
                try {
                    connection_pg.close();
                    connection_mysql.close();
                }
                catch (SQLException e) {
                    e.printStackTrace();
                    throw new RuntimeException(e);
                }
            }
        }
    }
}
```

8.3 补充习题

1. 填空题

- ① 嵌入式 SQL 语句为了和主语言语句进行区分，在 SQL 语句前加前缀_____，以_____结束。
- ② 主变量可以附加一个指示变量，指示变量可以表示输入主变量是否为_____。
- ③ SQL 是面向集合的，主语言是面向记录的，可以使用_____解决这一问题。
- ④ 存储过程经过编译、优化之后存储在_____。
- ⑤ 应用程序中访问和管理数据库的方法有_____、_____、_____、_____和 OLEDB 等。